3105. Longest Strictly Increasing or Strictly Decreasing Subarray

Daily Question - November 12, 2026

Problem Statement

You are given an array of integers nums. Return the length of the longest subarray of nums which is either strictly increasing or strictly decreasing.

Examples

Example 1: nums = [1,4,3,3,2]. Output: 2. The longest strictly increasing subarray is [1,4]; the longest strictly decreasing subarray is [4,3] or [3,2].

Example 2: nums = [3,3,3,3]. Output: 1. No adjacent pair is strictly increasing or strictly decreasing.

Example 3: nums = [3,2,1]. Output: 3. The entire array is strictly decreasing.

Constraints

1 <= nums.length <= 50; 1 <= nums[i] <= 50.

Solution: One-Pass Running Lengths

Maintain two running lengths: increasing, the length of the current strictly increasing run, and decreasing, the length of the current strictly decreasing run. Initialize both to 1 because every single element forms a valid subarray.

For each adjacent pair nums[i - 1] and nums[i]:

If nums[i] > nums[i - 1], increase the increasing run by 1 and reset the decreasing run to 1.

If nums[i] < nums[i - 1], increase the decreasing run by 1 and reset the increasing run to 1.

If they are equal, reset both runs to 1 because the subarray must be strictly monotonic.

After processing each pair, update the answer with the maximum of increasing and decreasing.

Complete Java Solution

```
class Solution {
    public int longestMonotonicSubarray(int[] nums) {
        int increasing = 1;
        int decreasing = 1;
        int answer = 1;

        for (int index = 1; index < nums.length; index++) {
            if (nums[index] > nums[index - 1]) {
                increasing++;
                decreasing = 1;
```

```
            } else if (nums[index] < nums[index - 1]) {
                decreasing++;
                increasing = 1;
            } else {
                increasing = 1;
                decreasing = 1;
            }

            answer = Math.max(
                    answer,
                    Math.max(increasing, decreasing)
            );
        }

        return answer;
    }
}
```

Correctness

At every index, increasing is exactly the length of the longest strictly increasing subarray ending at that index, and decreasing is exactly the length of the longest strictly decreasing subarray ending at that index. This follows directly from the transition rules: a continuation is allowed only when the adjacent relationship preserves the required strict direction.

Every nonempty subarray ends at some index, so taking the maximum of these two running lengths over all indices gives the length of the longest valid subarray. Equal adjacent values correctly reset both runs because neither strictly increasing nor strictly decreasing subarrays may contain equal adjacent elements.

Complexity

Time: O(n), since the array is scanned once. Auxiliary space: O(1), because only three integer variables are used.

Implementation Notes

Initialize answer, increasing, and decreasing to 1 to handle arrays of length 1 and arrays with no monotonic adjacent pair. The solution uses strict comparisons, so equal values break both current runs.